Generate final package

Below is your final package for the completed **Student & Course Management** project. It fits a Spring Boot + Thymeleaf CRUD application using Spring Data JPA with one-to-many / many-to-one mapping, Thymeleaf form binding, and the standard Controller → Service → Repository flow.[1] [2] [3]

Step-by-step summary

1. Create the Spring Boot project with dependencies for Spring Web, Thymeleaf, Spring Data JPA, and MySQL Driver. This stack supports server-side rendered CRUD pages backed by a relational database.[2] [1]
2. Configure `application.properties` with MySQL database name, username, password, port, JPA settings, and Thymeleaf defaults so the application can connect and render templates properly.[2]
3. Create the `Course` and `Student` entities with proper JPA mapping. In this project, one course can be linked to many students, while each student belongs to one course through a foreign key.[4] [3]
4. Build repository interfaces for both entities using Spring Data JPA so standard CRUD methods are available without writing repetitive DAO code manually.[1] [2]
5. Add service classes between controller and repository to keep the project structure clean and follow the Controller → Service → Repository pattern expected in academic Spring Boot projects.[1] [2]
6. Create controllers for course and student modules to handle list, create, update, and delete routes. Thymeleaf forms typically bind submitted data back into Spring MVC objects using model attributes, which is why the controller layer is central to form processing.[5] [6]
7. Create Thymeleaf templates for listing, adding, editing, and deleting data. For the student form, the course dropdown depends on a controller-provided list of courses, so courses must exist first before students can be linked meaningfully.[6] [1]
8. Test the application in the correct CRUD order: create courses, create students, read list pages, update student data, delete one student, then safely delete an unused course. Parent-child delete behavior must be checked carefully because foreign-key constraints can block deleting referenced parent records.[7] [1]

Submission-style report

Title

Student & Course Management System using Spring Boot, Thymeleaf, MySQL, and Spring Data JPA[2] [1]

Abstract

The Student & Course Management System is a web-based academic project developed using Spring Boot, Thymeleaf, Spring Data JPA, and MySQL. The application performs CRUD operations on two related entities, Student and Course, and demonstrates server-side rendering, relational database mapping, form handling, and layered architecture.[3] [1] [2]

Objective

The main objective of this project is to build a working end-to-end CRUD application that demonstrates how multiple related tables can be managed in a Spring Boot application using one-to-many and many-to-one relationships. The project also aims to show how Thymeleaf can be used to render forms and list pages dynamically from backend data.[3] [1]

Technologies used

Component	Technology
Backend	Spring Boot [1]
View layer	Thymeleaf [1]
Database	MySQL [2]
ORM / Persistence	Spring Data JPA, Hibernate [2] [4]
Architecture	Controller → Service → Repository [2]

Modules

- Course Management: add, view, update, and delete courses.[1]
- Student Management: add, view, update, and delete students.[1]
- Relationship Mapping: assign one course to many students using foreign-key mapping.[4] [3]
- Form Handling: use Thymeleaf forms to submit and edit entity data.[5] [6]

Database design

The project uses two main tables:

- `course`
- `student`[3]

The relationship is:

- One course can have many students.[3]
- Each student belongs to one course through a foreign key column in the student table.[4] [3]

Working flow

The application flow is:
Thymeleaf UI → Controller → Service → Repository → Database → UI. This structure helps separate presentation logic, business logic, and database access cleanly.[2] [1]

Features implemented

- Create new course records.[1]
- Create new student records linked to existing courses.[8] [6]
- Display course and student lists.[1]
- Edit student details and update linked course selection.[6]
- Delete student records.[1]
- Delete unlinked course records safely. Foreign-key-linked parent deletes may fail unless delete rules or cascade behavior are configured.[7] [4]

Testing performed

The application was tested in the following order:

1. Create course records.
2. Create student records linked to courses.
3. Read both list pages.
4. Update student data.
5. Delete one student.
6. Delete one unreferenced course.[7] [1]

All major CRUD functions worked successfully during testing, and the relationship mapping displayed correctly in the Thymeleaf UI.[1]

Outcome

The project successfully demonstrates CRUD operations, JPA entity relationships, Thymeleaf form handling, MySQL integration, and layered backend architecture. It is suitable for academic submission, project demo, and viva discussion.[6] [3] [1]

Viva questions and answers

1. What is the main purpose of this project?

This project is used to manage students and courses using CRUD operations with relational mapping between entities. It demonstrates how Spring Boot, Thymeleaf, MySQL, and Spring Data JPA work together in a full-stack Java web application.[2] [1]

2. Why did you use Spring Boot?

Spring Boot reduces configuration effort and makes it easier to build Java web applications quickly with embedded server support and ready integration for web, template, and data layers.[1]

3. Why did you use Thymeleaf?

Thymeleaf is a server-side template engine that works well with Spring MVC for rendering dynamic HTML pages and handling forms directly from backend model data.[6] [1]

4. What relationship exists between Student and Course?

The project uses a many-to-one relationship from Student to Course and a one-to-many relationship from Course to Student. One course can be linked with many students, while each student belongs to one course.[4] [3]

5. Where is the foreign key stored?

The foreign key is normally stored in the child table, which in this case is the student table, referencing the course table.[3]

6. Why do we create courses before students?

The student form usually needs a populated course dropdown so that a student can be linked to an existing course. Without course data, the related selection field has nothing useful to bind to.[8] [6]

7. What is Spring Data JPA used for?

Spring Data JPA simplifies database operations by providing repository-based CRUD methods, reducing the need to write repetitive DAO code manually.[2] [1]

8. What is the role of the service layer?

The service layer holds business logic and sits between controller and repository, making the code cleaner and easier to maintain.[2]

9. How does Thymeleaf form binding work?

Thymeleaf forms commonly bind to a model object using `th:object` and field mappings, while Spring MVC receives the submitted values through a model attribute in the controller.[5] [6]

10. Why can deleting a course sometimes fail?

Deleting a course can fail if a student record still references that course through a foreign key. In relational databases, parent row deletion is often blocked until dependent child records are removed or cascade behavior is configured properly.[7] [4]

11. What is CRUD?

CRUD stands for Create, Read, Update, and Delete, which are the four basic operations performed on stored data in most database-driven applications.[9]

12. What architecture did you follow?

The project follows the Controller → Service → Repository architecture, along with MVC principles for request handling, business logic, and data access separation.[2] [1]

Resume/project description

Built a **Student & Course Management System** using **Spring Boot, Thymeleaf, MySQL, and Spring Data JPA** with full **CRUD operations, one-to-many / many-to-one entity mapping, server-side rendered UI**, and **Controller → Service → Repository architecture**. Implemented relational database design, Thymeleaf form binding, linked dropdown selection, and end-to-end data flow for academic project demonstration.[6] [3] [1]

GitHub README content

Project name

Student & Course Management System [1]

Description

This project is a CRUD-based web application built using Spring Boot, Thymeleaf, MySQL, and Spring Data JPA. It manages students and courses and demonstrates one-to-many / many-to-one relationships with server-side rendered pages.[3] [1]

Features

- Add, view, update, and delete courses.[1]
- Add, view, update, and delete students.[1]
- Link students to courses using foreign-key mapping.[3]
- Display mapped course details in the student list.[8]

- Use Thymeleaf forms for create and update operations.[6]

Tech stack

- Java
- Spring Boot[1]
- Thymeleaf[1]
- Spring Data JPA[2]
- MySQL[2]
- Maven[2]

Project structure

- `controller` – handles HTTP requests and page navigation.[2]
- `service` – contains business logic.[2]
- `repository` – handles database operations through Spring Data JPA.[1]
- `entity` – contains JPA entity classes.[3]
- `templates` – contains Thymeleaf HTML pages.[1]
- `static/css` – contains CSS files for UI styling.[2]

How to run

1. Create a MySQL database.
2. Update database username, password, and URL in `application.properties`.
3. Run the Spring Boot application.
4. Open `http://localhost:8080/courses` to create courses first.
5. Open `http://localhost:8080/students` to create students linked to courses.[8] [1]

CRUD test order

1. Create courses.
2. Create students.
3. Read both list pages.
4. Update a student.
5. Delete a student.
6. Delete an unlinked course.[7] [1]

Learning outcomes

- Spring Boot project setup and MVC flow.[1]
- Thymeleaf form binding and template rendering.[6]
- MySQL integration with Spring Data JPA.[2]
- One-to-many and many-to-one entity mapping.[4] [3]
- CRUD implementation and relational data handling.[1]

Demo script

Use this short demo order during submission:

1. Open the course page and show saved courses.[1]
2. Open the student page and show mapped course names.[8]
3. Add or edit one student to demonstrate form submission and dropdown binding.[6]
4. Delete one student and refresh to show persistence.[1]
5. Explain that course deletion depends on whether any student still references that course through a foreign key.[7]

If you want, I can next generate a **very clean final viva sheet**, or a **one-page polished report answer** you can directly write in your record/submission.

⁂

1. https://www.baeldung.com/spring-boot-crud-thymeleaf
2. https://github.com/bezkoder/spring-boot-thymeleaf-example
3. https://www.geeksforgeeks.org/advance-java/jpa-one-to-many-mapping/
4. https://www.bezkoder.com/jpa-one-to-many/
5. https://www.youtube.com/watch?v=AkNqBoCYqAs
6. https://www.wimdeblauwe.com/blog/2021/05/23/form-handling-with-thymeleaf/
7. https://stackoverflow.com/questions/32939616/jpa-delete-fails-integrity-constraint-violation-foreign-key-no-action-data
8. https://www.wimdeblauwe.com/blog/2021/04/16/using-html-select-options-with-thymeleaf/
9. https://www.geeksforgeeks.org/java/spring-boot-crud-operations/
10. https://www.youtube.com/watch?v=Tr1ve88lKa0
11. https://www.studocu.com/in/document/sasi-institute-of-technology-and-engineering/java-programming/student-management-system-cs101-project-report-with-crud-operations/144269325
12. https://www.youtube.com/watch?v=CvDS6DltIno
13. https://www.youtube.com/watch?v=U7WDfoiodOg